

Veneroni Andrea 29669A - Relazione Progetto “Automi e Segnali”

Indice:

- Introduzione
 - Organizzazione progetto
 - Descrizione breve
- Strutture utilizzate
- Funzioni
 - Gerarchia funzioni
 - Descrizione e complessità funzioni
 - Considerazioni sulla progettazione
 - Annotazioni

INTRODUZIONE

ORGANIZZAZIONE PROGETTO:

file principale: - 29669A_veneroni_andrea.go

file secondario (da eseguire insieme al principale): - stack.go

DESCRIZIONE BREVE:

Per risolvere il problema assegnato sono state adoperate strutture dati e algoritmi al fine di ridurre il più possibile la complessità computazionale che il problema richiede.

Il programma si basa su una **DFS** implementata attraverso una **pila** per ridurre al massimo lo spazio esplorato, usando questa tecnica descritta in dettaglio successivamente, si ottiene una notevole riduzione dello spazio inutilmente esplorato rispetto un algoritmo **BFS** nel momento in cui il percorso minio esiste, mentre nel caso in cui non esista si ottiene lo stesso una riduzione di spazio inutilmente osservato rispetto un algoritmo **DFS** ma non quanto nel primo caso.

Il progetto è stato strutturato in modo da poter essere facilmente modificabile se si vogliono implementare più funzionalità, per questo motivo è estremamente modularizzato.

STRUTTURE UTILIZZATE

POINT:

struttura dati che rappresenta un punto in una griglia rappresentato attraverso le coordinate **x** e **y**

```
type Point struct {  
    x int  
    y int  
}
```

STACK:

struttura dati che rappresenta una pila, rappresentata attraverso **head**, puntatore al primo nodo della pila il quale contiene:

- **value**: la posizione di ogni passo effettuato dall'automa
- **actions**: una slice contenenti le possibili azioni future in ordine di rilevanza

- **counter_action**: contatore che indica quale azione è stata selezionata come prossima
- **next**: che è il puntatore al nodo successivo

```

type Stack struct {
    head *StackNode
}
type StackNode struct {
    value Point
    actions []Point
    counter_action int
    next *StackNode
}

```

OBSTACLE:

struttura dati che rappresenta un ostacolo nel piano a forma rettangolare quadrata o lineare descritto attraverso un **point** che indica il punto in basso a sinistra e un **point** che indica il punto in alto a destra

```

type obstacle struct {
    low Point
    high Point
}

```

AUTOMATON:

struttura dati che rappresenta un singolo automa nel piano descritto attraverso un **point** che indica la sua posizione attuale e uno **stack** che rappresenta un possibile percorso minimo (se esso esiste) dell'automata per raggiungere il punto di richiamo, oppure rimane vuoto (se il percorso minimo non esiste)

```

type automaton struct {
    pos Point
    path Stack
}

```

PIANO:

struttura dati che rappresenta il piano su cui possiamo aggiungere ostacoli e automi, descritti attraverso un puntatore a **slice** che contiene tutti gli ostacoli del mondo e una **map** che contiene tutti gli automi del mondo come valori e il nome di ciascun automa come chiave (usiamo una map di stringe poiché il suo costo di accesso è $O(1)$)

(usiamo un puntatore a slice e una mappa di puntatori ad automaton, in modo tale da avere un type floor che anche se passato per copia e modificato in locale, modifica il nostro piano direttamente)

```
type floor struct {  
    obstacles *[]obstacle  
    automaton map[string]*automaton  
}
```

FUNZIONI

GERARCHIA FUNZIONI:

```
main(){  
    newPiano() floor {}  
    esegui(p floor, s string){  
        crea(*floor){  
            newPiano() floor {}  
        }  
        parseInt(s string) int {}  
        stato(p *floor, x int, y int){  
            isAvailable (p *floor, obj any) bool {  
                isIn(target, poit_1, point_1 Point) bool {}  
            }  
        }  
        stampa(p *floor,){  
            posizioni(p *floor, w string){}  
        }  
        automa(p *floor, x int, y int, w string){  
            isAvailable (p *floor, obj any) bool {  
                isIn(target, poit_1, point_1 Point) bool {}  
            }  
        }  
        ostacolo(p *floor, a int, b int, c int, d int){  
            isAvailable (p *floor, obj any) bool {  
                isIn(target, poit_1, point_1 Point) bool {}  
            }  
        }  
        richiamo(p *floor, a int, b int, w string){  
            selectedAutomata(p *floor, w string) map[string]*automaton {}  
            findPath(p *floor, a int, b int, w string) {  
                obsInPath(p *floor, aut_pos, goal Point) []obstacle {  
                    isIn(target, poit_1, point_1 Point) bool {}  
                }  
            }  
        }  
        posizioni(p *floor, w string){  
            selectedAutomata(p *floor, w string) map[string]*automaton {}  
        }  
        esistePercorso(p *floor, a int, b int, w string){  
            findPath(p *floor, a int, b int, w string) {  
                obsInPath(p *floor, aut_pos, goal Point) []obstacle {  
                    isIn(target, poit_1, point_1 Point) bool {}  
                }  
            }  
        }  
        path(p *floor, obs []obstacle, aut *automaton, goal Point) (bool, Stack){
```


isAvailable(p *floor, obj any) bool {}:

complessità: $O(n)$ dove n = numero automi o numero ostacoli

funzione che presi in input o un ostacolo o un automa o un punto, verifica nel primo caso se nell'area dell'ostacolo che si vuole aggiungere è presente o meno un automa, nel secondo caso se l'automato che si vuole aggiungere è dentro l'area di un ostacolo, nel terzo se un punto assegnato è un punto disponibile oppure si trova dentro un ostacolo (in tutti i casi si usa la funzione isIn())

stato(p *floor, x int, y int){}:

complessità: $O(2n) = O(n)$ dove n = numero automi, il 2 indica la somma tra la sua complessità e quella di isAvailable()

verifica se la posizione data è disponibile (isAvailable ()) e scorre la map degli automi fino a quando non trova quello in quella posizione, se esiste indica che quel punto è un automa, se non esiste dice che quel punto è vuoto, se (isAvailable ()) non è verificata, dice che quel punto fa parte di un ostacolo

selectedAutomata(p *floor, w string) map[string]*automaton {}: complessità: $O(n)$ dove n = numero automi

restituisce una map di puntatori ad automi contenente tutti gli automi con un certo prefisso

posizioni(p *floor, w string){}:

complessità: $O(2n) = O(n)$ dove n = numero automi, il 2 indica la somma tra la sua complessità e quella di selectedAutomata()

stampa a schermo secondo il formato richiesto le posizioni di tutti gli automi dato un certo prefisso (invoca selectedAutomata())

stampa(p *floor,){}:

complessità: $O(3n) = O(n)$ dove n = numero automi, il 2 indica la somma tra la sua complessità e quella di selectedAutomata() e posizioni()

stampa a schermo secondo il formato richiesto le posizioni di tutti gli automi (invoca posizioni()) con una stringa vuota come argomento) e tutti gli ostacoli

automa(p *floor, x int, y int, w string){}: complessità: $O(n)$ complessità di isAvailable()

inserisce nel piano un automa data la sua posizione e il suo nome se e solo se non si trova in un ostacolo (verificato con isAvailable())

ostacolo(p *floor, a int, b int, c int, d int){}: complessità: $O(n)$ complessità di isAvailable()

inserisce nel piano un ostacolo data la sua posizione se e solo se nessun automa si trova in questo ostacolo (verificato con isAvailable())

obsInPath(p *floor, aut_pos, goal Point) []obstacle {}: complessità: $O(n)$ dove n = numero ostacoli

verifica quali sono gli ostacoli che hanno almeno un vertice all'interno dell'area del percorso minimo (il percorso minimo può essere formato solo attraverso uno spostamento effettuato dentro il rettangolo formato dai vertici dell'arrivo e dell'inizio) (utilizza isIn())

couldExist(p *floor, tot_obs []obstacle, start, goal Point) bool {}:

complessità: $O(2n) = O(n)$ dove n = numero ostacoli, il 2 indica la somma tra la sua complessità e quella di isAvailable()

verifica preliminare della possibile esistenza del percorso minimo, se il punto di arrivo fa parte di un ostacolo (usa isAvailable()) o un ostacolo taglia di netto in due l'area del percorso minimo, allora restituisce falso, altrimenti vero (il percorso minimo può esistere)

norm(n int) int {}: complessità: $O(1)$

normalizza la componente di un vettore

distanceToObsXorY(obs obstacle, aut Point, xORy, direction int) int {}:

complessità: $O(1)$

calcola la distanza la pos attuale e un ostacolo dato sul suo asse delle x o delle y, se questa distanza non esiste restituisce -1

chooseAction(obs []obstacle, aut *automaton, goal Point) []Point {}:

complessità: $O(n)$ dove n = numero ostacoli nel percorso

dati gli ostacoli presenti nell'area di percorso, la posizione dell'automa e il punto di arrivo, sceglie quale sia l'azione migliore da fare in quella situazione (possono essere solo due altrimenti non è più un percorso minimo). Per sceglierla, utilizza un'euristica basata sulla distanza da un ostacolo (distanceToObsXorY()) cercando di scegliere l'azione che lo tiene il più lontano possibile (se sull'asse delle x un ostacolo dista 12 e sull'asse delle y un altro dista 11, cercherà di muoversi sull'asse delle x) e la distanza dal punto di arrivo. Restituisce una slice di azioni in ordine di priorità

correctDirection(p *floor, path *Stack, obs []obstacle, aut *automaton, goal Point, call *int, stack_overflow *bool, was_there *map[Point]int){}:

complessità: funzione ricorsiva, calcolo complessità su singola esecuzione: $O(1) + O(n)$ costo di chooseAction() = $O(n)$

aumenta il counter (call) che sta a indicare quante volte è già stata chiamata e se esso non supera 100.000 (numero di sicurezza), inserisce nella pila dell'automa, la sua pos attuale e le azioni in ordine di esecuzione per il prossimo spostamento (chooseAction())
invoca la funzione doStep() illustrata successivamente per eseguire il passo verso il punto di arrivo.

Altrimenti termina l'esecuzione e settando a true `stack_overflow`, segnala che deve essere invocata nuovamente

`doStep(p *floor, path *Stack, obs []obstacle, aut *automaton, goal Point, call *int, stack_overflow *bool, was_there *map[Point]int){}`:

complessità: complessità: funzione ricorsiva, calcolo complessità su singola esecuzione: $O(n)$ (dovuta a `isAvailable()`)

se non siamo ancora arrivati al punto di arrivo, applica lo spostamento dell'automa in base all'azione in priorità,
se essa non lo fa rimanere fermo (nel caso sia sul limite del rettangolo del percorso minimo e l'azione migliore sarebbe uscire, l'azione migliore dice di stare fermi),
se la nuova posizione non è mai stata visitata prima (posizioni visitate in precedenza in `was_there`),
e se essa lo manda in una posizione disponibile (`isAvailable()`), invocherà di nuovo `correctDirection()` per aggiornare la posizione e procedere.
Altrimenti viene rimosso il primo nodo della pila (che indica il passo attuale) e viene invocato di nuovo `doStep()` sul passo precedente (notare che ogni volta che viene invocato `doStep()` il counter delle azioni eseguite aumento, ciò permette di passare all'azione che l'algoritmo reputava meno ottima per verificare se si sbagliava o meno, oppure se il counter sfiora dal numero di azioni disponibili, permette di fare backtracking fino a quando non si trova un'azione ottima che porta in un punto disponibile)

`path(p *floor, obs []obstacle, aut *automaton, goal Point) (bool, Stack){}`:

*complessità: nel caso peggiore richiamerà `correctDirection()` e poi `doStep()` per Area percorso minimo, quindi per un massimo di: $(l * h) * (O(n) + O(n)) = (l * h) * O(n)$*

ritorna un booleano che indica se il percorso esiste o meno, e l'eventuale pila contenente i dati del possibile percorso. Per farlo invoca `couldExist()` per verificare se è possibile che ci sia o meno il percorso minimo, se non c'è risparmia tempo restituendo false e una pila vuota (sta ad indicare che il percorso non esiste).

Altrimenti invoca `correctDirection()` per calcolare l'eventuale percorso (non è detto che ci sia).

Notare che `correctDirection()` è invocata per un numero limite di volte per evitare lo stack overflow, svuotato lo stack viene invocata nuovamente fino a che non trova l'eventuale percorso

`findPath(p *floor, a int, b int, w string) {}`:

*complessità: $(l * h) * O(n)$*

cerca quali sono gli ostacoli da prendere in considerazione nel calcolo del percorso minimo (`obsInPath()`) e invoca `path()` per calcolare l'eventuale percorso

esistePercorso(p *floor, a int, b int, w string){}

*complessità: $(l * h) * O(n)$*

stampa come da richiesta del progetto "SI" se il percorso esiste, "NO" se il percorso non esiste (invoca findPath(){} per capire se esiste o meno, dopo averlo invocato resetta la posizione dell'automa a quella di partenza)

richiamo(p *floor, a int, b int, w string){}

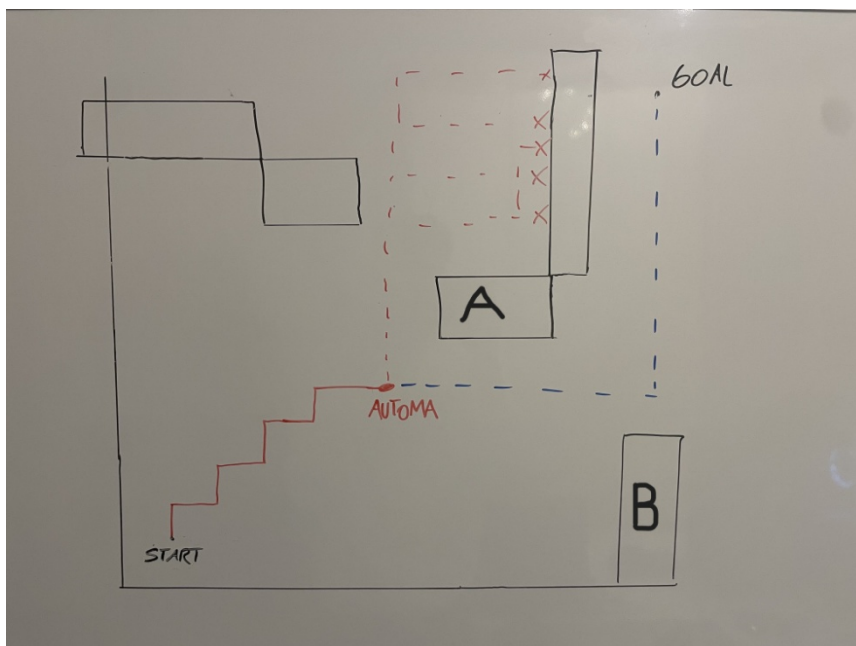
*complessità: $m((l * h) * O(n))$ dove n è il numero di ostacoli tra ogni automa e il goal e m è in numero di automi*

seleziona gli automi con il prefisso dato (selectedAutomata(){}) per poi farli muovere, se possibile, alla posizione assegnata (findPath(){})

CONSIDERAZIONI SULLA PROGETTAZIONE:

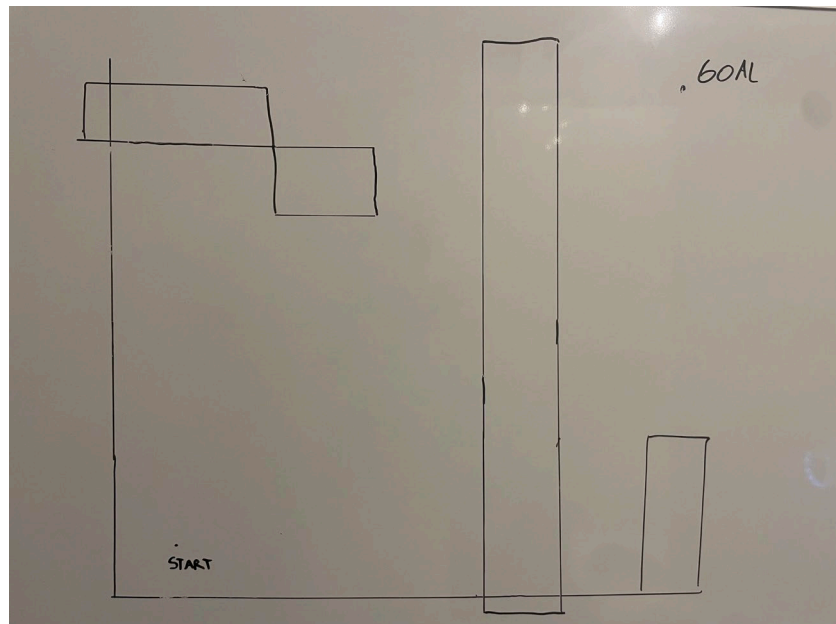
il modus operandi dell'algoritmo è estremamente efficiente come già detto nell'introduzione) se il percorso minimo esiste, infatti salvo rari casi, l'algoritmo tenderà a sbagliare azione molto raramente e non gli capiterà spesso di fare backtracking, inoltre anche se sbagliasse completamente percorso, grazie ad una mappa con memorizzate le posizioni già visitate dove si è dovuti tornare indietro (tempo di accesso $O(1)$), si riducono ancora di più le possibilità di errore da parte dell'algoritmo.

SITUAZIONE CRITICA:



in questo caso l'algoritmo non riesce a capire che la strada migliore non sia quella che procede sopra l'ostacolo "A", infatti grazie all'ostacolo "B" decide di andare verso l'alto (strada rossa) poiché sopra l'automa non sono presenti ostacoli, nonostante ciò raggiungerà comunque la soluzione in un tempo ragionevole non azzeccando però la strada migliore al primo colpo (indicata in blu)

SITUAZIONE CRITICA EVITATA:



In questo caso si ha una situazione critica potenziale evitata però a priori con una verifica preliminare, se si incontra questa casistica il costo computazionale per ottenere una risposta è praticamente nullo

Come considerazione finale, sotto sforzo con esempi dove il numero di ostacoli o gli spazi da percorrere erano molto elevati, l'algoritmo ha reagito benissimo andando in stack overflow controllato pochissime volte. Ho notato inoltre che il tempo computazionale non dipende affatto dal numero di ostacoli nel mondo salvo rari casi, e se l'area dello spazio percorribile aumenta di ordine n quadro, l'algoritmo risponde con un ordine di n . cosa che una comune **DFS** non farebbe.

ANNOTAZIONI:

- nel codice sono presenti alcuni commenti per le funzioni considerate da me più complicate, ignorarli se si reputano superflui
- nel codice è presente una funzione di test per leggere il percorso trovato